



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

공학석사학위논문

이중 프로세서 환경에서의 복수의
딥 러닝 어플리케이션 스케줄링 기법

2020년 8월

서울대학교 대학원
공과대학 컴퓨터공학부
오 진 우

요약

이종(Heterogeneous) 프로세서 환경에서의 복수의 딥 러닝 어플리케이션 스케줄링은 기본적으로 NP-난해(NP-Hard) 문제에 속하여 매우 큰 문제 공간을 가진다. 그래서 일반적으로 유전 알고리즘(GA, Genetic Algorithm) 같은 메타 휴리스틱(Meta-heuristic)이 사용된다. 그러나 메타 휴리스틱은 수행 시간이 매우 길어 런타임에 사용하기 어려운 단점을 지닌다. 따라서 본 논문은 이러한 메타 휴리스틱의 단점을 보완하면서 성능 또한 크게 떨어지지 않는 새로운 기법의 스케줄링 휴리스틱을 제안한다. 한편, 스케줄링 문제 해결에 일반적으로 활용되는 리스트 스케줄링(List Scheduling)은 복수 응용 스케줄링의 핵심인 스케줄 가능성(Schedulability)을 고려하지 못 한다. 이를 극복하기 위해 본 논문은 ‘합성과 반복 개선’ 방식을 개발하여 제안하는 휴리스틱을 구현하였다. 제안하는 휴리스틱의 성능을 입증하기 위해 CPU, GPU, NPU가 탑재된 실제 이종 프로세서 환경에서 여러 딥 러닝 네트워크들에 대한 실험을 진행하였다. 유전 알고리즘 스케줄러와 간단한 스케줄링 휴리스틱과의 성능 비교 실험을 통해, 제안하는 휴리스틱이 상대적으로 빠른 시간 내에 복수의 딥 러닝 어플리케이션에 대한 효과적인 스케줄링 결과를 도출하는 것을 검증하였다.

주요어 : 임베디드 시스템, 딥 러닝, 이종 프로세서, 스케줄링, 휴리스틱

학 번 : 2018-26822

목차

요약.....	i
그림 목차.....	iii
표 목차.....	v
1. 서론.....	- 1 -
2. 관련 연구.....	- 4 -
3. 스케줄링 모델.....	- 7 -
3.1 스케줄링의 대상.....	- 7 -
3.2 PE (Processing Element).....	- 8 -
3.3 프로파일링.....	- 9 -
3.4 스케줄 가능성 검사.....	- 10 -
3.4.1 최악 응답 시간.....	- 13 -
3.5 PEFT 알고리즘.....	- 13 -
4. 제안하는 휴리스틱 명세.....	- 16 -
4.1 1단계: 각 응용에 대한 PEFT 알고리즘 적용.....	- 17 -
4.2 2단계: 단순 합성 (Simple Synthesis).....	- 18 -
4.3 3단계: 반복 개선 (Iterative Improvement).....	- 19 -
5. 실험.....	- 23 -
5.1 이중 프로세서 환경에서의 복수(2개) 응용 스케줄링.....	- 25 -
5.2 이중 프로세서 환경에서의 복수(3개) 응용 스케줄링.....	- 27 -
6. 결론.....	- 32 -
참고 문헌.....	- 34 -

그림 목차

[그림 1] SqueezeNet 구조.....	- 7 -
[그림 2] DenseNet40_32 구조.....	- 7 -
[그림 3] 프로파일링 결과 예시	- 9 -
[그림 4] 최대 허용 간섭(MAD)의 초기화와 DBF 계산.....	- 10 -
[그림 5] 네트워크 의존성에 의한 최대 허용 간섭 갱신.....	- 11 -
[그림 6] OCT 계산 원리.....	- 14 -
[그림 7] JHeuristic 의사 코드.....	- 16 -
[그림 8] PEFT 알고리즘에 의한 딥 러닝 네트워크의 매핑 형태 분류.....	- 17 -
[그림 9] 단순 합성.....	- 18 -
[그림 10] 반복 개선 의사 코드.....	- 20 -
[실험 1] 스케줄러 별 각 응용(2개) WCRT 비교 (1).....	- 25 -
[실험 2] 실험 1에 대한 스케줄러 별 수행 시간 비교.....	- 25 -
[실험 3] 스케줄러 별 각 응용(2개) WCRT 비교 (2).....	- 27 -
[실험 4] 실험 3에 대한 스케줄러 별 수행 시간 비교.....	- 27 -
[실험 5] 스케줄러 별 각 응용(3개) WCRT 비교 (1).....	- 28 -
[실험 6] 실험 5에 대한 스케줄러 별 수행 시간 비교.....	- 28 -
[실험 7] 스케줄러 별 각 응용(3개) WCRT 비교 (2).....	- 29 -
[실험 8] 실험 7에 대한 스케줄러 별 수행 시간 비교.....	- 29 -
[실험 9] 스케줄러 별 각 응용(3개) WCRT 비교 (3).....	- 30 -
[실험 10] 실험 9에 대한 스케줄러 별 수행 시간 비교.....	- 31 -
[식 1] DBF로 계산한 높은 우선 순위 태스크의 간섭.....	- 11 -

[식 2] 응용에 대한 스케줄 가능성 판단 식.....	- 12 -
[식 3] 응용의 최악 응답 시간 계산 식.....	- 13 -
[식 4] OCT 계산식.....	- 14 -

표 목차

[표 1] 쿼드 코어 CPU에 대한 PE 세분화.....	- 9 -
[표 2] 가능한 실험 조합.....	- 24 -

1. 서론

최근 딥 러닝(Deep Learning)을 이용한 응용들은 더 이상 유행의 차원이 아니라 필수적으로 자리매김 하고 있어 딥 러닝 응용이 사용되지 않는 경우를 보기 힘들다. 이에 따라 딥 러닝 응용을 가속화하여 수행 시간을 줄이기 위한 여러 기술들이 소프트웨어/하드웨어적으로 활발히 연구되어 왔다. 특히, 하드웨어적으로는 딥 러닝 응용의 수행 시간을 단축시키기 위한 여러 프로세서들이 개발되어 디바이스에 탑재되고 있다. 따라서 기존의 동종(Homogeneous) 프로세서 환경에서 이종 (Heterogeneous) 프로세서 환경으로 변화하고 있는 추세다. 멀티코어 CPU와 GPU를 비롯하여 딥 러닝 응용을 빠른 시간 내에 실행할 수 있는 NPU(Neural Processing Unit), DSP(Digital Signal Processing) array 같은 프로세서들이 그 대표적인 예시다.

하지만 딥 러닝 응용을 수행하는 대부분의 플랫폼들은 일반적으로 디바이스에 탑재된 단일 고성능 프로세서만을 사용함으로써 빠른 딥 러닝 응용 수행 시간을 보장받는다. 즉, 내장된 여러 프로세서 모두를 동시에 활용하지 않는다. 한편, 디바이스 성능에 대한 요구사항 수준이 날이 갈수록 높아지면서 단일 딥 러닝 응용만을 수행하는 경우는 찾아보기 힘들다. 대표적인 사례로 객체 탐지(Object Detection), 영상 분할(Image Segmentation), 이미지 분류(Image Classification) 등의 딥 러닝 응용들을 동시에 수행해야 하는 자율 주행 자동차, 드론(Drone) 같은 디바이스들이 있다. 결론적으로, 복수의 딥 러닝 응용들을 수행하는 데 단일 고성능 프로세서만을 사용하는 전략은 비효율적이며 최적의 스케줄링을 보장할

수 없다. 따라서 여러 다른 프로세서들을 가정하는 이중 프로세서 환경에서의 복수의 딥 러닝 어플리케이션 스케줄링이 요구된다.

이중 프로세서 환경에서의 스케줄링에 관하여 기존에 여러 연구들이 선행되었다. 하지만 거의 대부분 리스트 스케줄링(List Scheduling) 방식을 기반으로 하기 때문에 복수 응용과 그에 대한 스케줄 가능성을 가정하지 않는 한계를 보인다. 복수 응용 스케줄링은 각 응용마다 실행 주기(Period)와 우선순위(Priority)를 가정한다. 같은 매핑이라도 주어진 주기와 우선순위에 따라 스케줄 가능 여부가 결정된다. 더불어 복수 응용 스케줄링은 응용들이 각기 다른 시작 시간(Dynamic offset)을 가질 수 있음을 가정한다. 따라서 응용의 실행 주기, 우선순위 그리고 시작 시간을 고려하여 매핑에 대한 스케줄 가능성을 필히 검사해야 한다.

최적의 스케줄링을 찾는 것은 대표적인 NP-난해(NP-Hard) 문제로서 큰 문제 공간을 탐색해야 한다. 여기에 스케줄링 해야 할 태스크의 개수와 프로세서의 개수가 많아진다면 문제 공간의 크기는 기하급수적으로 증가한다. 이렇게 큰 문제 공간을 탐색하는 데에는 여러 기법들이 적용될 수 있는데, 그 중 대표적인 기법으로 유전 알고리즘(GA, Genetic Algorithm)이 있다. 유전 알고리즘은 생물의 진화 과정에 착안한 기법이다. 모집단으로부터 여러 자식 해들을 만들고 해마다 부여된 적합값(Fitness)에 따라 이를 계승하거나 버린다. 이렇듯 기본적으로 무작위의 성격을 지니기 때문에 넓은 문제 공간을 탐색하는 데 적절하여 많은 NP-난해 문제에 응용된다. 하지만 수행 시간이 큰 단점을 지니기 때문에 실제 어플리케이션 운용에서 큰 제약을 보일 수 있다. 예를 들어, 여러 응용들이 긴박한 주기로 수행되는 상황에서는 빠른 시간 내에 효율적인 스케줄을 찾아야 하기 때문에 소요 시간이 막대한 유전 알고리즘은

현실적으로 사용하기 힘들다. 유전 알고리즘을 짧은 시간 동안만 수행하여 스케줄링 결과를 얻을 수도 있지만 그로부터 얻은 스케줄링이 좋을 가능성은 매우 희박하다. 따라서 문제 공간을 좀 더 효율적으로 탐색하기 위한 방법론이 필요하다.

본 논문에서는 지금까지 언급한 필요성과 여러 한계점들을 극복하여 이중 프로세서 환경에서의 복수의 딥 러닝 어플리케이션을 위한 새로운 스케줄링 기법을 제안한다. 이 기법은 유전 알고리즘 스케줄러보다 성능이 크게 뒤쳐지지 않는 스케줄을 찾아냄과 동시에 수행 시간은 월등히 빠른 장점을 지닌다.

2. 관련 연구

본 논문에서 다루는 스케줄링의 대상은 딥 러닝 네트워크로서, 이는 보다 일반적인 형태인 방향성 비순환 그래프(DAG, Directed Acyclic Graph)로 표현될 수 있다. 방향성 비순환 그래프는 정점들간의 간선이 방향성을 지니며(Directed) 네트워크 내에 사이클이 없는(Acyclic) 그래프로 정의된다. 이미지를 입력으로 받아 여러 중간층들을 거쳐 이미지의 유형을 추론하는 CNN(Convolutional Neural Network)도 이러한 구조로 이루어져 있다. 네트워크를 구성하는 각 층(Layer)들을 하나의 정점으로, 그리고 각 층들의 출력이 그 다음 층의 입력으로 들어가는 것을 하나의 간선으로 대응시킬 수 있기 때문이다. 한편, 방향성 비순환 그래프처럼 CNN을 비롯한 딥 러닝 네트워크는 한 정점에서 여러 정점으로 간선들이 이어지는 병렬성(Parallelism)을 지닐 수 있으며, 반대로 여러 정점으로부터 하나의 정점으로 간선이 이어지는 형태를 가질 수도 있다.

이러한 방향성 비순환 그래프는 정점이 하나의 태스크로, 그리고 간선이 태스크들 간의 의존성(Graph dependency)으로 대응될 수 있기 때문에 스케줄링 대상으로 고려될 수 있다. 방향성 비순환 그래프 스케줄링에 대한 여러 연구들이 선행되었다. HEFT(Heterogeneous Earliest Finish Time) [1] 알고리즘은 성격이 각기 다른 프로세서들이 존재하는 이종 환경에서의 태스크 스케줄링 알고리즘이다. 이는 방향성 비순환 그래프 상의 어느 한 정점으로부터 마지막 정점까지의 임계 경로(Critical Path)를 계산한 랭크(rank) 개념을 도입하였다. 먼저 각 태스크마다 랭크를 계산하여 태스크의 우선순위를 부여하는 우선 순위 할당(Task Prioritization) 단계를 진행한다.

그리고 높은 우선순위의 태스크부터 차례대로 EST(Earliest Start Time)을 최소화하는 방향으로 프로세서 선정(Processor Selection) 단계를 진행한다. 최종적으로 EFT(Earliest Finish Time)를 최소화하는 방향으로 알고리즘이 진행되며, 동시에 낮은 시간 복잡도를 갖는다.

CPOP(Critical Path On a Processor)[1] 알고리즘은 HEFT 알고리즘을 일부 변형한 형태로서, 그래프 상의 한 정점에서 마지막 정점까지의 임계 경로를 의미하는 랭크의 개념뿐만 아니라 해당 정점에서 시작 정점까지의 패스를 추가적으로 계산한 새로운 지표로 알고리즘을 구현하였다.

HEFT 알고리즘과 CPOP 알고리즘은 모두 EFT를 최소화하는 방향으로 스케줄링 한다는 점에서 효율적이다. 하지만 복수 응용을 가정하지 않는다는 점, 그리고 이중 프로세서 환경임에도 불구하고 랭크 계산에 쓰이는 태스크 수행 시간을 각 프로세서 수행 시간의 평균으로 사용하는 제한을 지닌다.

PEFT(Predict Earliest Finish Time)[2] 알고리즘은 HEFT[1], CPOP[1] 알고리즘처럼 이중 프로세서 환경을 가정하고 우선 순위 할당 단계와 프로세서 선정 단계로 구성된다. 하지만 각 단계에서 HEFT, CPOP 알고리즘과는 다른 새로운 지표를 정의하여 사용한다. 즉, 우선 순위 할당 단계에서는 각 태스크와 프로세서 쌍마다 OCT(Optimistic Cost Table) 값을 정의 및 계산하고 프로세서 선정 단계에서는 OCT 값과 EFT 값을 더한 OEFT 값을 정의 및 계산한다. HEFT 알고리즘의 랭크 값은 각 프로세서 수행 시간의 평균값이 사용되는 반면, PEFT 알고리즘의 OCT 값은 각 프로세서 마다의 수행 시간들이 모두 별도로 고려된다. 결론적으로 PEFT 알고리즘은 HEFT 알고리즘을 비롯한 다른 여러 알고리즘들보다 더 작은 EFT 값을 지니는 스케줄링 결과를 산출한다. 하지만 PEFT 알고리즘 역시 복수

응용을 가정하지 않다는 점에서 한계를 지닌다.

이 외에 HCPT[3], HPS[4], PETS[5]와 같이 여러 스케줄링 알고리즘들이 개발되었다. 하지만 이들 또한 이중 프로세서 환경을 고려할 뿐, 복수 응용 스케줄링을 다루지 않고 단일 응용만을 가정한다.

논문 [6]은 이중 프로세서 환경에서의 복수의 방향성 비순환 그래프들에 대한 스케줄링 알고리즘을 제시한다. 해당 논문의 핵심은 독립적인 복수의 방향성 비순환 그래프들을 하나의 큰 그래프로 묶은 후에 기존에 존재하는 단일 응용 스케줄링 알고리즘을 적용하는 것이다. 하지만 이는 서로 다른 실행 주기를 지니는 복수의 그래프들에 대해서 적용할 수 없을 뿐만 아니라, 각 그래프들이 실행되는 시점(Dynamic offset)에 대한 고려가 되어 있지 않다.

마지막으로 논문 [7]은 복수의 방향성 비순환 그래프들에 대한 스케줄링 알고리즘을 제시함과 동시에 스케줄 가능성에 대해서도 고려하고 있다. 하지만 동종(Homogeneous) 프로세서 환경만을 가정하여 제한이 있다.

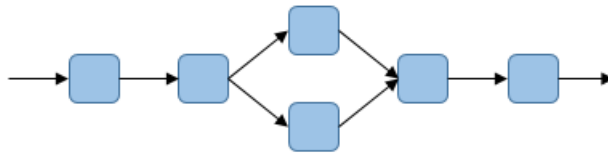
3. 스케줄링 모델

본 절에서는 본 논문이 가정하는 스케줄링 환경과 제안하는 휴리스틱에 적용되는 여러 배경 지식들을 소개한다.

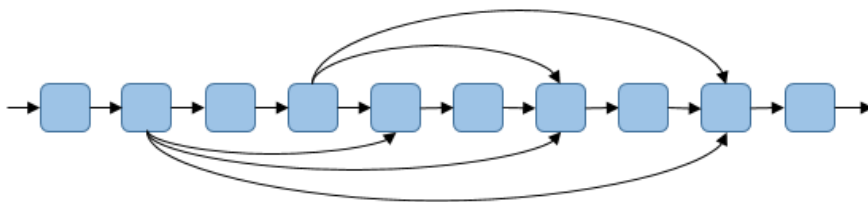
3.1 스케줄링의 대상

본 논문은 스케줄링의 대상으로 SqueezeNet[8], MobileNet_v1[9], MobileNet_v2[10], DenseNet40_32[11]의 총 4가지 딥 러닝 네트워크들을 다룬다. 제안하는 휴리스틱이 일부 네트워크 형태에서만 효과를 보이는 것이 아니라, 일반적으로 효과를 보인다는 점을 보이기 위해 보다 여러 형태의 네트워크들을 고려하였다.

SqueezeNet은 병렬적인 구조를 갖는 대표적인 딥 러닝 네트워크이다 (그림 1). 병렬성을 지니는 네트워크에 대한 스케줄링이 가능함을 보이기



[그림 1] SqueezeNet 구조



[그림 2] DenseNet40_32 구조

위해 본 네트워크를 선정하였다. MobileNet_v1과 MobileNet_v2는 병렬성이 없는 구조를 갖는 네트워크이다. MobileNet 계열의 네트워크들은 모바일 기기에서 많이 사용되기 때문에 임베디드 디바이스 상에서의 스케줄링을 다루는 본 논문과 관련이 깊다. 마지막으로 DenseNet40_32는 그림 2와 같이 하나의 레이어에서 여러 다른 레이어로 간선들이 연결되며 사슬처럼 엮이는 형태를 갖는 네트워크로서, 지금까지 소개한 다른 네트워크들보다 복잡한 구조를 지닌다. 단순히 병렬구조 혹은 직렬 구조뿐만 아니라 복잡한 구조의 네트워크에 대해서도 제안하는 휴리스틱이 잘 동작한다는 것을 보이기 위해 본 네트워크를 선정하였다.

3.2 PE (Processing Element)

딥 러닝 네트워크의 레이어는 스케줄링에서의 태스크와 동일하다. 본 논문에서는 태스크가 매핑될 수 있는 PE로 CPU, GPU, NPU를 선정하였다. 특징적으로, 본 논문에서는 CPU를 단순히 하나의 PE로 보는 것이 아니라 내부에 존재하는 코어 단위로 세분화했다. 이를 통해 코어 단위로 레이어를 매핑할 수 있고 레이어 간 병렬성(Inter-layer parallelism)을 높일 수 있다. 본 논문은 삼성 갤럭시 S9에 탑재된 big.LITTLE CPU(4x Samsung Exynos M3 2.7GHz, 4x ARM Cortex-A55 1.79GHz)를 사용하였다. big.LITTLE CPU의 특성상, 딥 러닝 응용과 같이 연산량이 많은 응용은 big CPU(4x)가 활성화되어 사용된다. 따라서 표 1과 같이 쿼드 코어를 가정하여 CPU를 여러 PE로 세분화하였다.

예를 들어 표 1의 분류 (3, 1)의 경우, CPU가 2개의 PE로 세분화되며 각 PE에서 사용하는 CPU 코어의 개수는 각각 3개, 1개라는 의미이다. 즉, 어떤 레이어가 전자에 매핑되면 3개의 코어가 동작하여 수행되고 후자에

[표 1] 쿼드 코어 CPU에 대한 PE 세분화

분류	(4)	(3, 1)	(2, 2)	(2, 1, 1)	(1, 1, 1, 1)
PE 개수	1개	2개	2개	3개	4개

매핑되면 1개의 코어가 동작하여 수행된다. PE마다 할당되는 코어 개수가 다르기 때문에 각 PE에서의 레이어 수행 시간에도 차이가 발생한다.

3.3 프로파일링

삼성 갤럭시 S9에는 ARM 계열의 big.LITTLE CPU와 GPU(ARM Mali-G72)가 탑재되어 있다. 그래서 ARM 계열 CPU, GPU에 최적화된 머신러닝 라이브러리인 ACL(ARM Compute Library)[12]을 사용하여 CPU, GPU 상에서 레이어 수행 시간을 측정하였다. 또한, 그래프 의존성이 존재하고 서로 다른 PE에 매핑된 두 레이어 간의 통신 시간에 대한 프로파일링도 진행하였다. 한편, 삼성 갤럭시 S9에는 NPU가 탑재되어 있지 않기 때문에 NPU의 속도를 big CPU(4x) 대비 6.5배 빠른 것으로 산정하였다 [13][14].

그림 3과 같이 프로파일링 정보는 Caffe 플랫폼에서 사용하는 prototxt 파일 형식으로 저장되어 스케줄러에 사용된다. 각 레이어마다 이름(name), 수행 시간 단위(time_unit) 그리고 각 PE에서의 레이어 수행 시간이 저장되어 있다. 그림 3은 표 1의 분류 (4)에 해당되는 프로파일링 결과이다.

```

layer {
  name: "c1"
  time_unit: "usec"
  cpu: 3951.30004883
  gpu: 1701.0
  npu: 607.892333984
}

layer {
  name: "p3"
  time_unit: "usec"
  cpu: 422.0
  gpu: 294.0
  npu: 64.9230804443
}

layer {
  name: "s1"
  time_unit: "usec"
  cpu: 114.300003052
  gpu: 64.0
  npu: 17.5846157074
}

```

[그림 3] 프로파일링 결과 예시

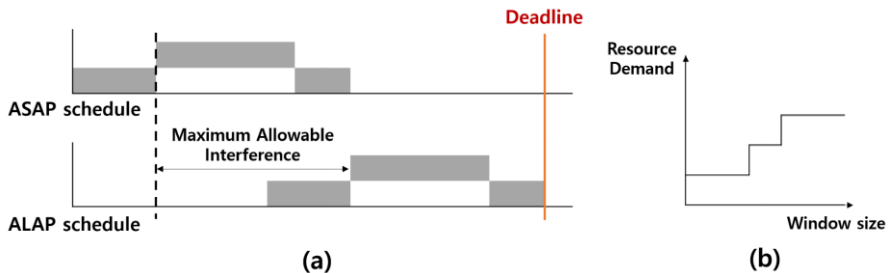
즉, CPU를 하나의 PE로 고려하기 때문에 그림 3에서 볼 수 있듯이 하나의 CPU 필드만 존재한다. 만약 표 1의 분류 (2, 1, 1)의 경우라면, CPU 필드가 cpu1~cpu3까지 총 3개 존재할 것이다. 그 때의 각 필드는 순서대로 2개, 1개, 1개 코어로 동작한 프로파일링 결과이다.

CPU를 여러 PE로 세분화함에 따라 여러 특이점들이 발견되었다. 첫째는 DVFS(Dynamic Voltage Frequency Scaling)에 따라 CPU frequency가 실시간으로 변동하는 현상이다. 그리고 둘째는 CPU 활용도(Utilization)가 낮은 경우에 Hot-Plugging 옵션에 따라 일부 CPU 코어가 비활성화(Off)되는 현상이다. 본 논문은 이러한 하드웨어적 현상들을 모두 고려하여 레이어 프로파일링을 진행하였다.

3.4 스케줄 가능성 검사

복수 응용들을 프로세서에 매핑하여 스케줄링 할 때, 스케줄이 각 응용의 우선 순위와 주기를 만족하는지 검사하는 것을 스케줄 가능성 검사라고 한다. 스케줄을 실행할 때 이러한 제한 사항을 만족하지 못한다면 상황에 따라 부정적인 결과를 초래할 수 있다. 따라서 스케줄 가능성 검사는 복수 응용 스케줄링에서 필수적이다.

먼저, 최대 허용 간섭(MAI, Maximum Allowable Interference)[15]은 발생



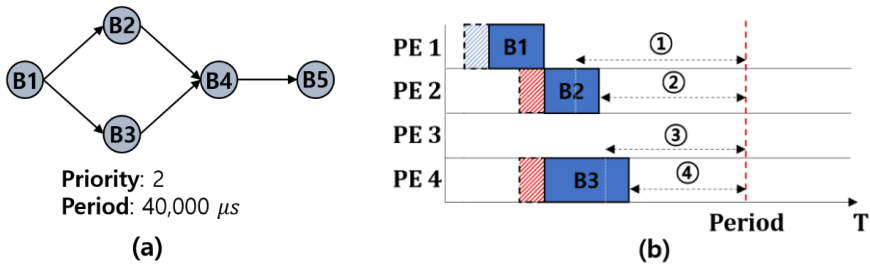
[그림 4] 최대 허용 간섭(MAI)의 초기화와 DBF 계산

가능한 여러 간섭들에 의해서 태스크가 최대로 밀려날 수 있는 여유 시간을 의미한다. 스케줄 가능성을 검사하기 위해 먼저 최대 허용 간섭을 초기화 한다. 응용의 모든 태스크가 프로세서에 매핑된 스케줄이 있을 때, 그림 4(a)와 같이 가장 빨리 실행될 수 있는 ASAP(As-Soon-As-Possible) 스케줄을 가정한다. 그리고 해당 응용이 실행 주기까지 최대로 밀려나 가장 늦게 실행되는 ALAP(As-Late-As-Possible) 스케줄을 가정한다. 이 때, 최대 허용 간섭은 이러한 두 스케줄 간의 시간 차이로 초기화된다.

이렇게 초기화된 여유 시간을 발생 가능한 여러 간섭들을 고려하여 갱신해나간다. 첫째로, 어떤 태스크의 실행 시간과 여유 시간(MAI)을 더한 시간 동안에는 같은 PE에 매핑된 높은 우선 순위 태스크들이 자원을 요구하여 간섭을 일으킬 수 있다. DBF(Demand Bound Function)(그림 4(b))를 통해 이 간섭의 최대치를 계산할 수 있다(식 1). 둘째로, 이러한 높은 우선 순위 태스크의 간섭은 같은 PE의 태스크에만 영향을 주지 않고 네트워크 의존성으로 연결된 다른 PE의 태스크도 간섭할 수 있다. 예를 들어,

[식 1] DBF로 계산한 높은 우선 순위 태스크의 간섭

$$\sum_{hp \in H(T_i)} DBF_{M(T_i)}(hp, MAI_{T_i} + C_i)$$



[그림 5] 네트워크 의존성에 의한 최대 허용 간섭 갱신

그림 5(a)와 같이 응용 B가 있다고 가정하자. 그리고 응용 B는 우선 순위가 2로서, 우선 순위가 1인 다른 응용에 의해 간섭을 받는다고 하자. 그래서 그림 5(b)처럼 태스크 B1이 파란색 빗금만큼 미뤄져 최대 허용 간섭이 줄어든다고 하자. 한편, 그림 5(a)에서 태스크 B1과 네트워크 의존성을 갖는 태스크는 B2와 B3이다. 이 태스크들은 PE 1이 아닌 다른 PE에 매핑되어 있음에도 불구하고(그림 5(b)), 태스크 B1와의 의존성으로 인해 태스크 B1에서 발생한 높은 우선 순위의 간섭을 물려받는다. 따라서 그림 5(b)처럼 빨간색 빗금만큼 태스크 B2와 B3의 최대 허용 간섭이 줄어들게 된다. B2의 최대 허용 간섭은 그림 5(b)의 ①에서 ②로 줄어들고, B3의 최대 허용 간섭은 ③에서 ④로 줄어든다.

위처럼 네트워크의 첫 번째 레이어부터 마지막 레이어까지의 최대 허용 간섭을 네트워크 의존성에 따라 갱신해나간다. 이로써 마지막 레이어의 최대 허용 간섭은 선행 레이어들의 간섭들을 모두 고려한 응용 전체의 최종적인 여유 시간이 된다. 한편, 본 논문은 비선점(Non-preemptive) 스케줄링을 가정한다. 그래서 만약 높은 우선 순위 태스크가 실행될 때 낮은 우선 순위 태스크가 이미 실행되고 있었다면, 높은 우선 순위 태스크는 실행을 대기해야 한다. 최악의 경우, 낮은 우선 순위 태스크들 중 가장 큰 수행 시간만큼 대기해야 한다. 결론적으로, 마지막 레이어의 여유 시간(응용의 여유 시간)에서 이러한 낮은 우선 순위 태스크의 간섭을 뺀다. 그 값이 양수면 스케줄이 가능하고, 음수면 스케줄이 불가능하다 (식 2).

[식 2] 응용에 대한 스케줄 가능성 판단 식

$$MAI_{last_layer} - \max_{lp \in L(last_layer)}(C_{lp}) \geq 0$$

[식 3] 응용의 최악 응답 시간 계산 식

$$Period_{App} - MAI_{last_layer} + \max_{lp \in L(last_layer)}(C_{lp})$$

3.4.1 최악 응답 시간

단일 응용 스케줄링은 응용의 수행 시간을 줄이는 방향으로 동작한다. 이와 달리 복수 응용 스케줄링은 스케줄 가능성, 즉 여러 간섭들을 보수적으로 고려해야 하기 때문에 각 응용의 최악 응답 시간(WCRT, Worst Case Response Time)을 줄이는 방향으로 수행되어야 적절하다.

3.4절에서 설명한 과정대로 응용의 모든 레이어에 대한 최대 허용 간섭(MAD)을 갱신하면, 마지막 레이어의 최대 허용 간섭은 선행 레이어들의 간섭들을 모두 고려한 응용 전체의 여유 시간이 된다. 이 마지막 레이어의 최대 허용 간섭(응용의 여유 시간)을 응용의 주기에서 뺀다. 그리고 마지막 레이어가 매핑된 PE에서 최악의 경우로 발생할 수 있는 낮은 우선 순위 태스크의 간섭을 더하면 응용의 최악 응답 시간을 계산할 수 있다(식 3).

3.5 PEFT 알고리즘

본 논문은 기존에 존재하는 단일 응용 스케줄링 기법을 응용하여 복수 응용 스케줄링 문제를 해결한다. 관련 연구의 여러 스케줄링 알고리즘 중 비교적 좋은 성능을 내는 것으로 알려진 PEFT(Predict Earliest Finish Time) [2] 알고리즘을 응용한다.

PEFT 알고리즘의 핵심은 태스크-프로세서 쌍마다 OCT(Optimistic Cost Table) 값을 계산하여 태스크에 우선순위를 부여하고 태스크의 매핑을 결정하는 것이다. 본 절에서는 이러한 OCT 값이 구체적으로 어떻게 계산되는지

[식 4] OCT 계산 식

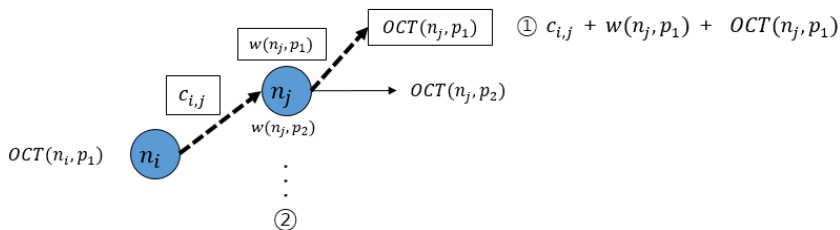
$$OCT(n_i, p_k) = \frac{\textcircled{2}}{\textcircled{1}} \frac{\max_{n_j \in succ(n_i)} [\min_{p_w \in P} \{OCT(n_j, p_w) + w(n_j, p_w) + \overline{c_{i,j}}\}]}{\overline{c_{i,j}} = 0 \text{ if } p_w = p_k},$$

살펴본다.

태스크와 PE 쌍에 대한 OCT 값은 위의 식 4로 계산된다. n 은 태스크, p 는 매핑되는 PE, w 는 PE에서의 태스크의 수행 시간, c 는 네트워크 의존성으로 연결된 두 태스크 간의 통신 시간을 의미한다. 식 4의 ①과 ②를 도식화하면 그림 6과 같다.

식 4의 ①을 통해 그림 6의 점선으로 표현된 경로가 가장 작은 값으로 계산되어 선택되었다고 가정하자. 그 값은 그림 6의 네모 박스로 표시된 것들을 모두 더한 값이다. 가장 작은 값으로 선택된 것이기 때문에 이는 EFT(Earliest Finish Time)를 고려, 스케줄링의 성능을 높이게 된다.

식 4의 ①은 나머지 후임(succeeding) 태스크들에 대해서도 이뤄진다 (그림 6의 ②). 그러면 모든 후임 태스크들에 대한 그림 6의 ① 값이 계산되고, 그 중에 가장 큰 값이 최종적인 OCT 값이 된다. 이는 일반적인 스케줄링



[그림 6] OCT 계산 원리

휴리스틱이 그러하듯, 임계 경로(Critical Path)인 것에 더 높은 우선 순위를 부여하는 것과 유사한 전략이다.

하나의 태스크와 모든 PE 쌍에 대해서 위와 같은 방식으로 OCT를 계산한다. 그리고 OCT 값들의 평균값을 그 태스크의 우선순위로 한다. 그리고 이 평균값에 EFT를 더한 값이 가장 작은 프로세서로 해당 태스크를 매핑한다.

4. 제안하는 휴리스틱 명세

PEFT 알고리즘을 비롯한 관련 연구의 스케줄링 알고리즘들은 대부분 리스트 스케줄링 방식을 따른다. 하지만 이 방식은 복수 응용 스케줄링에 적용될 수 없다. 첫째, 스케줄 가능성 검사는 모든 태스크의 매핑이 정해진 상태를 가정하는 반면 리스트 스케줄링은 태스크를 우선 순위에 따라 하나씩 매핑하기 때문에 매핑 과정에서 스케줄 가능성 검사를 적용할 수 없다. 둘째, 리스트 스케줄링은 단일 응용의 수행 시간을 줄이는 데 집중하기 때문에 복수 응용들 간의 간섭을 고려한 응용의 최악 응답 시간을 계산하지 못 한다. 한편, 유전 알고리즘과 같은 메타 휴리스틱은 매우 큰 수행 시간을 소요하여 런타임에 사실상 적용이 불가능하다. 본 논문은 이러한 한계점들을 극복하기 위해 ‘합성과 반복 개선’이라는 기법을 적용한 새로운 스케줄링 휴리스틱을 제안한다. 이를 JHeuristic이라고 칭하며, 이는 총 3단계(그림 7의 ‘PEFT_algorithm’, ‘simple_synthesis’, ‘iterative_improvement’)로 구성된다.

```
function JHeuristic {  
    for each application:  
        PEFT_algorithm(application)  
    simple_synthesis()  
    for each application:  
        iterative_improvement(application)  
    return solutions  
}
```

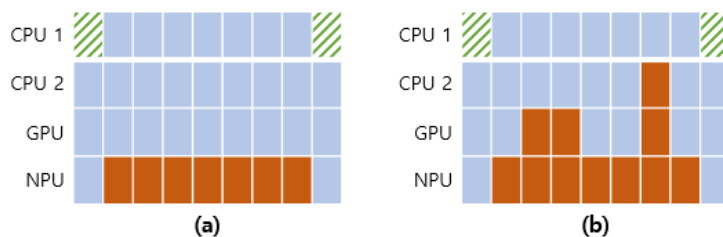
[그림 7] JHeuristic 의사 코드

4.1 1단계: 각 응용에 대한 PEFT 알고리즘 적용

본 단계에서는 각 응용마다 PEFT 알고리즘을 적용하여 응용 별로 수행 시간이 최소화된 스케줄을 알아낸다. 이를 통해 탐색할 문제 공간을 초기화한다.

수행 시간을 최소화하는 스케줄은 가장 빠른 PE 자원을 주로 사용하게 되는데, 그에 따라 그림 8과 같이 크게 두 가지 형태로 스케줄 패턴이 분류될 수 있다. 이후의 2, 3단계에서 이러한 스케줄 패턴을 고려하여 문제 공간을 탐색한다.

MobileNet_v1은 복잡한 구조나 병렬성이 하나도 없는, 순수하게 1차 형태의 직렬 구조를 지닌 네트워크다. 직렬 구조의 특성상 어떤 레이어를 처리할 때 그 다음 레이어는 해당 레이어가 끝나 출력값을 넘겨줄 때까지 기다려야만 한다. 다시 말해, 직렬 구조 네트워크의 경우 동시에 수행할 수 있는 레이어들이 존재할 수 없고, 한 번에 레이어 하나씩만 실행할 수 있다. 따라서 MobileNet_v1에 PEFT 알고리즘을 적용한다면, 그림 8(a)처럼 가장 빠른 PE(NPU)만을 이용하여 EFT를 최소화하는 스케줄링 형태가 나오게 된다.



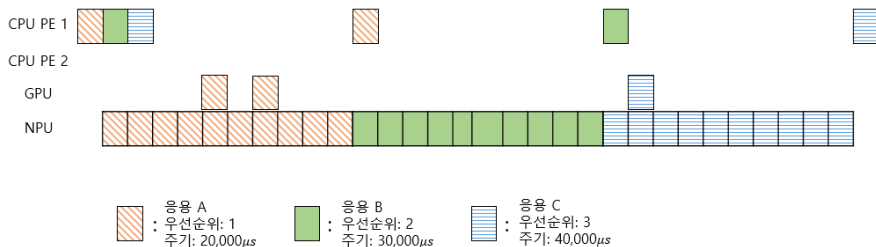
[그림 8] PEFT 알고리즘에 의한 딥 러닝 네트워크의 매핑 형태 분류

이에 반해 SqueezeNet은 하나의 레이어에서 2개의 간선이 뻗어 나와 각기 다른 레이어로 이어지는 구조를 포함한다. 이 2개의 레이어들은 이전 레이어의 수행이 끝나고 나면 병렬적으로 수행될 수 있다. 따라서 이러한 SqueezeNet에 PEFT 알고리즘을 적용한다면, 그림 8(b)처럼 여러 레이어가 같은 시간대에 실행되는 형태를 볼 수 있다. 하지만 이러한 형태는 오직 병렬적인 부분에서만 발생하고, 대부분의 경우는 마찬가지로 직렬 구조이기 때문에 결국 가장 빠른 PE가 주로 사용되는 형태가 나타난다.

그림 8에서 빗금으로 표시되어 있는 맨 앞의 레이어(frontend layer)와 맨 뒤의 레이어(backend layer)는 각각 입력을 읽고 결과를 출력하는 레이어다. 이는 범용 프로세서가 처리해야 하는 부분이므로 그림 8과 같이 반드시 CPU PE에 매핑되어야 한다.

4.2 2단계: 단순 합성 (Simple Synthesis)

본 단계는 1단계에서 나온 각 응용의 스케줄들을 합쳐서 복수 응용 스케줄링처럼 만든다. 그림 9는 3개의 응용에 대한 단순 합성 예시다. 복수 응용 스케줄링이기 때문에 각 응용마다 실행 우선순위와 실행 주기가



[그림 9] 단순 합성

부여된다. 이로써 모든 태스크들의 매핑이 결정되기 때문에 복수 응용에 대한 스케줄 가능성 검사도 가능하게 된다.

그림 9의 스케줄은 각 응용의 입장에서 볼 때 수행 시간이 최소화된 스케줄이지만, 복수 응용 스케줄링 차원에서 볼 때는 매우 비효율적인 스케줄이다. 대부분의 레이어들이 같은 PE에 매핑되어 서로 극단적으로 간섭을 주고 받기 때문이다. 이 상태에서는 짧은 최악 응답 시간은 물론, 스케줄 가능성도 기대할 수 없다. 따라서 다음 3단계에서 문제 공간을 탐색한다.

4.3 3단계: 반복 개선 (Iterative Improvement)

본 단계는 2단계에서 나온 스케줄을 변형한다. 즉, 빠른 PE에 매핑된 높은 우선 순위의 레이어들을 다른 PE로 이동시키면서 문제 공간을 탐색한다. 이는 높은 우선 순위 응용이 갖고 있던 빠른 PE 자원을 다른 응용에게 조금씩 양보해 나가는 역할을 한다. 그리하여 극단적으로 간섭을 받고 있던 낮은 우선 순위 응용들의 최악 응답 시간을 줄이고 스케줄 가능성을 높여 나간다. 이에 따라 높은 우선 순위 응용의 최악 응답 시간은 점차 늘어나지만, 주어진 주기를 만족하는 내에서 PE 자원을 낮은 우선순위 응용에게 최대한 양보하는 방향으로 나아갈 것이다.

반복 개선은 높은 우선 순위 응용에서 낮은 우선 순위 응용 순으로, 그리고 첫 번째 레이어에서 마지막 레이어 순으로 가장 빠른 PE에 집중 매핑되어 있는 레이어들을 다른 PE로 분산시킨다. 이 때, 한 번에 몇 개의 레이어를 옮길 것인지 결정해야 한다. 의사코드(그림 10)에서 그룹 단위(chunk_unit)는 옮기려는 레이어의 개수를 의미한다. 예를 들어, 그룹 단위가 5라면 한 번에 다섯 개의 레이어들을 다른 PE로 옮겨본다는 뜻이다.

```

function iterative_improvement(app) {
    chunk_unit = [ 1, 5, 10,  $\frac{\text{len}(\text{app})}{2}$ , len(app) ]
    progress = 0
    ① while progress < len(app):
        ② for chunk in chunk_unit:
            ③ for each PE:
                simulate chunk moving to each PE
                compare with previous_mapping
                get the best PE
            ④ if the best PE is potential:
                move chunk to the best PE
                update previous_mapping
                progress += chunk
                break and repeat while loop
            ⑤ else:
                try next chunk unit
                if no next chunk unit:
                    try next app
        try next app
    }

```

[그림 10] 반복 개선 의사 코드

그림 10의 for문 ②에서 알 수 있듯이, 그룹 단위는 1부터 시작해서 최대 네트워크 길이까지 시도된다. 만약 레이어 1개(그룹 단위 1)를 다른 PE로 옮겨보고 개선이 없다면, 그 다음 그룹 단위인 5개의 레이어를 옮겨본다. 여기서 만일 개선이 발생한다면 레이어 이동을 확정하고, 남은 레이어들에 대해서 다시 그룹 단위 1부터 레이어 이동을 시도한다. 그룹 단위를 1부터 설정하는 이유는 네트워크에 병렬성이 있다면 그를 최대한 보존하기 위함이다. 그리고 그룹 단위를 네트워크 길이까지 설정하는 이유는 지역 최적(Local Optimum)에서 벗어나 적극적으로 문제공간을 탐색하기 위함이다.

그룹 단위를 정했다면, 어느 PE로 그룹을 옮길 것인지 결정해야 한다. 그림 10의 for문 ③을 통해, 다른 PE로 그룹을 옮겨 볼 때마다 각 응용의

최악 응답 시간을 저장한다. 그리고 이를 이전 매핑에서의 각 응용의 최악 응답 시간과 비교한다. 만약 그룹을 옮겨본 매핑에서의 응용의 최악 응답 시간이 이전 매핑에서의 응답 시간보다 줄어 들었다면 이득을 본 것으로 판단한다. 그래서 그 변화 값에 1을 곱하여 저장한다. 반대로 그룹을 옮겨본 매핑에서의 응용의 최악 응답 시간이 이전 매핑에서의 응답 시간보다 늘어났다면 손해를 본 것으로 판단한다. 그래서 그 변화 값에 -1을 곱하여 저장한다. 이렇게 각 응용에 대한 변화 값에 1 혹은 -1을 곱하여 저장한 후, 이들을 모두 더한 값을 PE마다 계산한다. 그리고 그 값이 가장 큰 PE를 가장 좋은 PE(그림 10의 ‘the best PE’)로 결정한다.

하지만 가장 좋은 PE로 그룹을 옮기는 것이 항상 적절하다고 볼 수 없다. 예를 들어, 가장 좋은 PE로 그룹을 옮겼을 때 응용의 최악 응답 시간 변화 값에 곱해진 숫자(1 또는 -1)들이 모두 -1이라고 가정해보자. 이는 그룹을 옮겼을 때 모든 응용의 최악 응답 시간이 늘어난다는 것을 의미하기 때문에 이러한 방향으로 문제 공간을 탐색하는 것은 적절하지 않다. 그래서 이 경우에는 잠재력(potential, 그림 10의 ④)이 없다고 정의한다. 반대로 만약 하나의 응용이라도 숫자 1을 가지고 있다면, 그 가장 좋은 PE는 문제 공간 탐색에 있어 잠재력을 지닌다고 정의한다. 결론적으로, 가장 좋은 PE가 잠재력이 있다면(개선이 발생 한다면) 그룹을 해당 PE로 옮긴다. 그리고 남은 레이어에 대해서 반복 개선을 시도한다(그림 10의 ①). 반대로 가장 좋은 PE가 잠재력이 없다면(개선이 없다면) 그 다음으로 큰 그룹 단위에 대해서 반복 개선을 시도한다 (그림 10의 ②). 만약 그룹 단위를 네트워크 길이까지 모두 시도해봤다면, 해당

응용에 대한 반복 개선을 종료하고 그 다음 응용에 대해 반복 개선을 시도한다.

개선이 일어날 때마다 매핑에 대해 스케줄 가능성을 검토한다. 그리고 스케줄이 가능한 매핑이라면 최종 해에 이를 추가한다. 그리하여 파레토 최적(Pareto Optimal) 해를 구한다. 이는 하나의 해만 얻는 대부분의 휴리스틱과 차별성이 있다고 볼 수 있다.

5. 실험

JHeuristic과의 성능 비교 대상으로 유전 알고리즘(GA, Genetic Algorithm)과 단순 휴리스틱(Simple Heuristic)을 선정하였다. 유전 알고리즘은 대표적인 메타 휴리스틱으로서 최적에 근사한 스케줄을 찾을 수 있지만 수행 시간이 매우 큰 단점을 지닌다. 따라서 성능과 수행 시간의 차원에서 JHeuristic과의 비교 대상으로 설정 하였다. 단순 휴리스틱은 가장 문제가 될 수 있는 레이어를 다른 PE로 옮기는 과정을 반복하며 문제 공간을 탐색하는 간단한 휴리스틱이다. JHeuristic의 성능 비교 하한선으로 설정하였다. 이를 보다 구체적으로 설명하면, 단순 휴리스틱은 일단 frontend, backend 레이어를 제외한 모든 레이어들을 가장 빠른 PE에 매핑한다. 그리고 나서 가장 문제가 될 수 있는 레이어를 선택하여 다른 PE로 옮기는 과정을 반복한다. 여기서 ‘가장 문제가 될 수 있는’ 레이어는 스케줄이 안 되는(간섭에 의해 최대 허용 간섭이 음수가 되는) 레이어, 혹은 그 레이어보다 우선 순위가 낮으면서 수행 시간이 가장 큰 레이어, 혹은 그 레이어보다 우선 순위가 높으면서 간섭이 가장 큰 레이어 중에 하나로 결정된다.

본 실험 절에서는 기본적으로 JHeuristic이 스케줄 가능한 매핑 결과를 잘 주는지 살펴본다. 그리고 JHeuristic을 통해 얻은 스케줄의 성능(최악 응답 시간)과 JHeuristic의 수행 시간이 어느 정도인지 살펴본다. 그리고 문제 공간의 크기와 실행 주기의 변화에 따른 JHeuristic의 성능과 수행 시간을 관찰한다. 그래서 JHeuristic이 빠른 시간 내에 효율적인 스케줄을 산출할 수 있음을 검증한다.

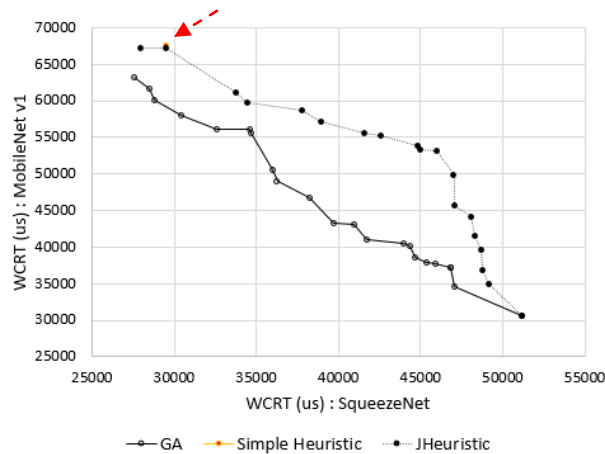
표 2는 여러 가지 가능한 실험 조합을 보여준다. 어떤 응용을 쓸 것인지, 몇 개의 응용을 스케줄링 할 것인지, 어떤 이중 프로세서 환경을 쓸 것인지, CPU 세분화는 어떻게 할 것인지 그리고 각 응용의 주기는 얼마나 잡을 것인지에 따라 각 실험을 진행할 수 있다. 본 실험 절에서는 이 중에서 대표적으로 몇 가지 조합을 선정하여 실험 결과를 제시한다.

[표 2] 가능한 실험 조합

응용의 종류	SqueezeNet
	MobileNet v1
	MobileNet v2
	DenseNet40_32
응용의 개수	2개
	3개
이중 프로세서 환경	CPU, GPU
	CPU, GPU, NPU
CPU 세분화	(4)
	(3, 1)
	(2, 2)
	(2, 1, 1)
	(1, 1, 1, 1)
응용의 주기	여유
	긴박

이후의 각 실험 결과의 제목에서, 네트워크 이름 다음의 괄호 안에는 해당 네트워크의 실행 우선순위와 실행 주기가 적혀있다. 실행 우선순위는 ‘pr’ (priority)로 표기하였으며, 숫자가 작을수록 우선순위가 높음을 의미한다. 실행 주기는 ‘p’ (period)로 표기하였으며, 주기의 단위는 μs (micro-second)이다.

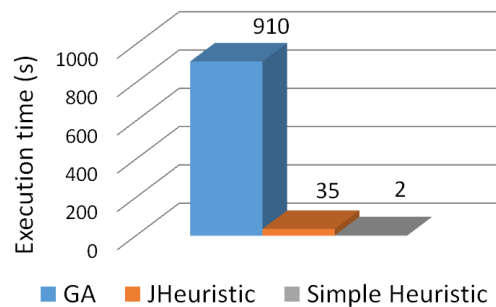
5.1 이종 프로세서 환경에서의 복수(2개) 응용 스케줄링



[실험 1] 스케줄러 별 각 응용(2개) WCRT 비교 (1)

SqueezeNet (pr=1, p=60,000us), MobileNet v1 (pr=2, p=70,000us)

이종 프로세서 환경: CPU 세분화 (3, 1) + GPU 1 주기: 여유

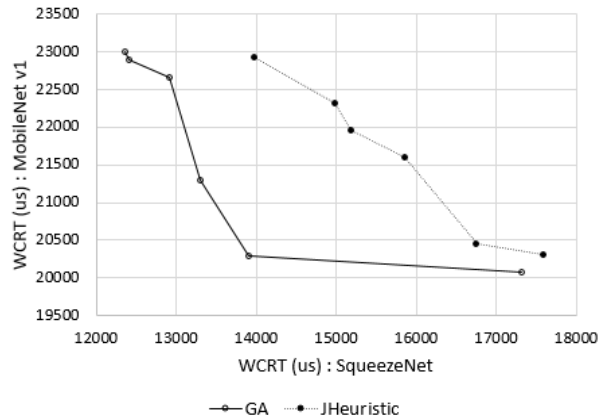


[실험 2] 실험 1에 대한 스케줄러 별 수행 시간 비교

실험 1에서 JHeuristic은 높은 우선 순위인 SqueezeNet의 최악 응답 시간이 늘어나는 대신 낮은 우선 순위인 MobileNet_v1의 최악 응답 시간은 줄어드는 파레토 최적 결과들을 보여준다. 이 결과들은 모두 스케줄 가능한 해들이다. 여러 개의 해들이 출력되는 이유는 앞서 언급하였듯이 반복 개선 단계에서 레이어들을 다른 곳으로 옮길 때마다 매핑이 스케줄 가능하다면 최종 해로 저장해놓기 때문이다.

유전 알고리즘의 파레토 최적 그래프는 JHeuristic의 파레토 최적 그래프보다 조금 더 아래에 위치한다. 따라서 응용의 최악 응답 시간 차원에서 유전 알고리즘의 성능이 JHeuristic의 성능보다 더 좋다고 할 수 있다. 하지만 실험 2의 결과를 보면 유전 알고리즘의 수행 시간(910초)은 JHeuristic의 수행 시간(35초)의 약 26배가되는 것으로 나타났다. 실험 1에서 보이는 스케줄의 최악 응답 시간 차이에 비해 두 스케줄러 간의 수행 시간의 차이가 매우 크다. 비록 JHeuristic은 최악 응답 시간 차원에서 유전 알고리즘만큼 좋은 스케줄링 결과를 보여주진 못 하지만, 월등한 수행 시간 차이를 감안한다면 현실적으로 적용 가능한 스케줄러는 유전 알고리즘보다는 오히려 JHeuristic이라 말할 수 있다. 한편, 단순 휴리스틱의 결과는 JHeuristic의 결과보다 살짝 위쪽에 위치한다(실험 1의 빨간색 점선 화살표). 그리고 파레토 최적 결과들을 제시하는 JHeuristic과는 달리 단 하나의 매핑 결과만을 보여주는 한계를 보인다.

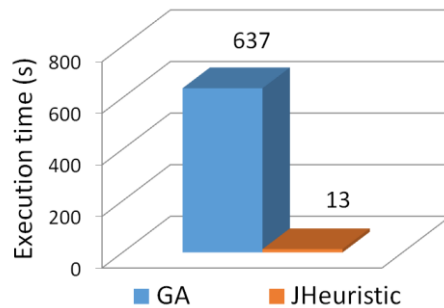
실험 3, 실험 4는 이전의 실험 1, 실험 2보다 더 긴박한 주기를 가정하고 NPU가 추가된 실험이다. 긴박한 주기에서도 유전 알고리즘과 JHeuristic의 결과를 관찰할 수 있지만 단순 휴리스틱은 스케줄 결과를 보여주지 못 한다. 실험 결과에 대한 분석은 실험 1, 실험 2에 대한 분석과 유사하다.



[실험 3] 스케줄러 별 각 응용(2개) WCRT 비교 (2)

SqueezeNet (pr=1, p=18,000us), MobileNet v1 (pr=2, p=23,000us)

이중 프로세서 환경: CPU 세분화 (3, 1) + GPU + NPU | 주가: 긴박

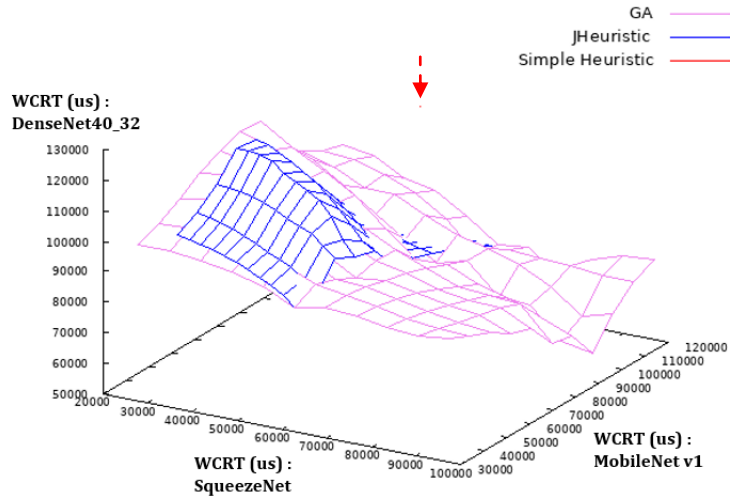


[실험 4] 실험 3에 대한 스케줄러 별 수행 시간 비교

5.2 이중 프로세서 환경에서의 복수(3개) 응용 스케줄링

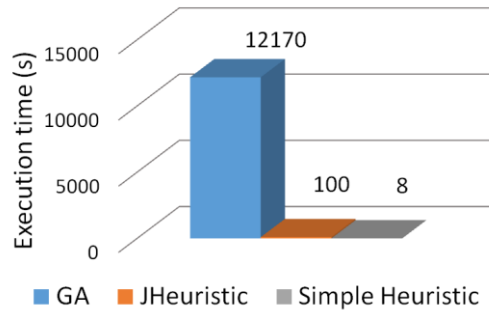
응용의 개수를 늘려, 본 실험 절에서는 3개 딥 러닝 네트워크 스케줄링에 대한 실험을 진행하였다. 이전까지의 실험 결과와 뼈대는 동일하지만, 응용 개수가 늘어남에 따라 2차원 그래프가 아닌 3차원 그래프로 결과가 표현된다는 점이 특징적이다.

실험 5에서 유전 알고리즘의 결과가 JHeuristic의 결과보다 약간 아래에



[실험 5] 스케줄러 별 각 응용(3개) WCRT 비교 (1)

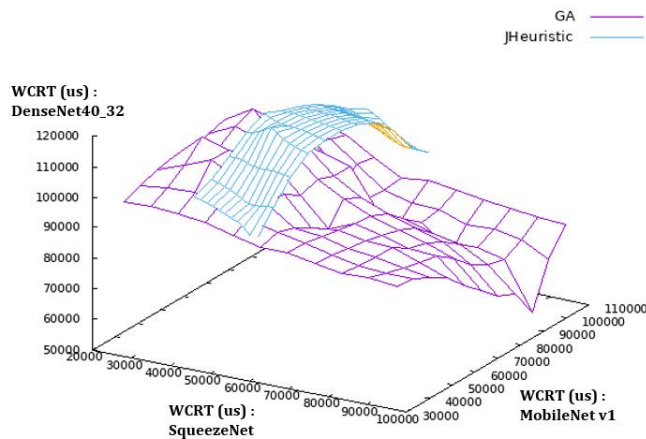
SqueezeNet (pr=1, p=110,000us), MobileNet v1 (pr=2, p=120,000us), DenseNet40_32 (pr=3, p=130,000us) ! 이중 프로세서 환경: CPU 세분화 (2, 1, 1) + GPU ! 주기: 여유



[실험 6] 실험 5에 대한 스케줄러 별 수행 시간 비교

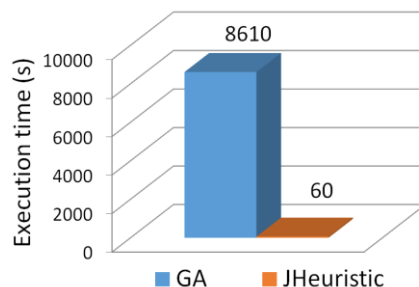
위치하지만, 이전의 실험들보다 두 그래프가 더 가까이 붙어있음을 알 수 있다. 즉, 최악 응답 시간 차원에서 두 스케줄러 간의 성능 차이가 크게 나지 않는다. 하지만 유전 알고리즘의 수행 시간은 응용의 개수가 2개에서 3개로 늘어나면서 이전보다 급격하게 늘어났다. 기존의 실험 2와 실험 4에서의 유전 알고리즘의 수행 시간은 각각 910초와 637초였지만,

실험 6에서는 무려 12,170초(약 202분)로 나타났다. 이렇듯 유전 알고리즘의 수행 시간은 응용의 개수가 증가할 때마다(문제 공간이 커질수록) 막대하게 증가할 것이다. 반면 JHeuristic은 이러한 경우에도 100초의 시간만이 소요되었다. 결론적으로, 응용의 개수가 증가하면서 유전 알고리즘과 JHeuristic의 수행 시간 격차는 심해지지만 각 응용별 최악 응답 시간 차이는 상대적으로 크게 나지 않는다. 한편, 단순 휴리스틱의 결과는 두 개의 평면 그래프보다 월등히 위쪽에 위치한다(실험 5의 빨간색 점선 화살표).



[실험 7] 스케줄러 별 각 응용(3개) WCRT 비교 (2)

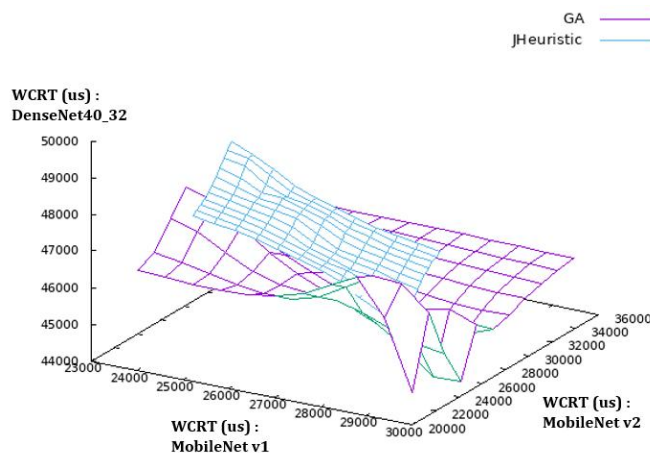
SqueezeNet (pr=1, p=100,000us), MobileNet v1 (pr=2, p=110,000us), DenseNet40_32 (pr=3, p=120,000us) | 이중 프로세서 환경: CPU 세분화 (2, 1, 1) + GPU | 주기: 긴박



[실험 8] 실험 7에 대한 스케줄러 별 수행 시간 비교

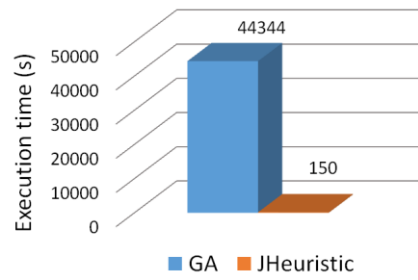
실험 7, 실험 8은 이전의 실험 5, 실험 6에서 주기를 더 긴박하게 설정한 실험이다. 커진 문제 공간과 긴박한 주기로 인해 단순 휴리스틱은 스케줄 결과를 보여주지 못 한다. 실험 결과에 대한 분석은 실험 5, 실험 6에 대한 분석과 유사하다.

마지막으로 실험 9와 실험 10은 NPU가 추가되고 SqueezeNet 대신 MobileNet_v2를 대상으로 진행되었다. MobileNet_v2는 본 논문에서 대상으로 하는 네트워크들 중에 가장 많은 수의 레이어들로 구성되는 네트워크다. 따라서 이전보다 훨씬 큰 문제 공간을 만들어낸다. 실험 결과를 통해, 응용의 최악 응답 시간 차원에서 유전 알고리즘의 결과와 JHeuristic의 결과가 서로 밀접하게 붙어있지만 각 스케줄러의 수행 시간 차이는 이전보다 더 커졌다는 것을 알 수 있다. 그 결과로 유전 알고리즘 수행에 44,344초(약739분)가 소요되었다. 이는 지금까지의 실험 중에서 가장 큰 값이다. 반면 JHeuristic은 150초가 소요되었다. 한편, 단순 휴리스틱은 커진 문제 공간으로 인해 결과를 내지 못 하였다.



[실험 9] 스케줄러 별 각 응용(3개) WCRT 비교 (3)

MobileNet v1 (pr=1, p=30,000us), MobileNet v2 (pr=2, p=40,000us), DenseNet40_32 (pr=3, p=50,000us) | 이중 프로세서 환경: CPU 세분화 (2, 1, 1) + GPU + NPU | 주기: 긴박



[실험 10] 실험 9에 대한 스케줄러 별 수행 시간 비교

6. 결론

지금까지 본 논문은 이중 프로세서 환경에서의 복수의 딥 러닝 응용 스케줄링 문제를 다루었다. 이는 관련 연구에 소개된 기존 연구들에서는 다루지지 않았던 주제였다. 유전 알고리즘과 같은 메타 휴리스틱은 이러한 스케줄링 문제에 대해 좋은 성능을 보여주긴 하지만, 수행에 상당한 시간이 걸리는 치명적인 단점을 지닌다. 이를 극복하여 본 논문에서는 유전 알고리즘의 성능에 크게 뒤처지지 않으면서 빠른 시간 내에 스케줄 결과를 줄 수 있는 새로운 기법의 휴리스틱(JHeuristic)을 제안하였다. 또한 기존의 단일 응용 스케줄링에 많이 응용되는 리스트 스케줄링의 한계점을 극복하여 복수 응용 스케줄링을 가능케 하였다. 최종적으로 유전 알고리즘과 단순 휴리스틱과의 비교 실험을 통해 JHeuristic의 성능을 입증하였다.

딥 러닝 응용에 대한 수요가 점점 높아짐에 따라, 앞으로 이중 프로세서 환경에서 복수의 딥 러닝 응용들이 운용될 가능성이 높아지고 있다. 따라서 이에 필요한 좋은 스케줄링을 본 논문에서 제안하는 JHeuristic을 통해 빠르게 얻을 수 있을 것으로 기대된다.

본 논문에서 제안하는 JHeuristic은 유전 알고리즘과 같은 메타 휴리스틱에 비해 월등히 빠른 속도로 효율적인 스케줄을 찾을 수 있지만, 메타 휴리스틱만큼 좋은(최악 응답 시간이 최소화된) 스케줄을 찾는 데는 여전히 한계가 있다. 따라서 향후 연구를 통해 스케줄러의 입력 패턴을 보다 정형화하여 탐색할 문제 공간을 효율적으로 압축할 수 있을 것으로 보인다. 이에 따라 문제 공간을 탐색하는 알고리즘 또한 최적화하여 더

좋은 성능의 스케줄을 찾아내는 스케줄러로 발전시킬 수 있을 것으로
사료된다. 더 나아가, 스케줄의 실효성을 보이기 위해 JHeuristic으로 얻은
스케줄을 실제 디바이스에서 검증해볼 필요성이 대두된다.

참고 문헌

- [1] Topcuoglu, Haluk, Salim Hariri, and Min-you Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, Vol. 13, No. 3, pp. 260-274, 2002.
- [2] Arabnejad, Hamid, and Jorge G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Transactions on Parallel and Distributed Systems*, Vol. 25, No. 3, pp. 682-694, 2013.
- [3] Hagra, Tarek, and Jan Janecek, "A simple scheduling heuristic for heterogeneous computing environments," *Proceedings of the Second international conference on Parallel and distributed computing*, pp. 104-110, 2003.
- [4] Ilavarasan, E., P. Thambidurai, and R. Mahilmanan, "High performance task scheduling algorithm for heterogeneous computing system," *International conference on algorithms and architectures for parallel processing*, pp.193-203, Springer, Berlin, Heidelberg, 2005.
- [5] Ilavarasan, E., and P. Thambidurai, "Low complexity performance effective task scheduling algorithm for heterogeneous computing environments," *Journal of Computer sciences*, Vol. 3, No. 2, pp.94-103, 2007.
- [6] Zhao, Henan, and Rizos Sakellariou, "Scheduling multiple DAGs onto heterogeneous systems," *Proceedings 20th IEEE International Parallel &*

Distributed Processing Symposium, pp. 14, 2006.

[7] Saifullah, Abusayeed, et al., "Parallel real-time scheduling of DAGs," *IEEE Transactions on Parallel and Distributed Systems*, Vol. 25, No. 12, pp. 3242-3252, 2014.

[8] Iandola, Forrest N., et al., "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size," *arXiv preprint arXiv:1602.07360*, 2016.

[9] Howard, Andrew G., et al., "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[10] Sandler, Mark, et al., "Mobilenetv2: Inverted residuals and linear bottlenecks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp.4510-4520, 2018.

[11] Huang, Gao, et al., "Densely connected convolutional networks," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp.4700-4708, 2017.

[12] ARM. (2017). ARM Compute Library. Available: <https://developer.arm.com/technologies/compute-library>

[13] Andrei Frumusanu. (2018). HiSilicon Kirin 970 - Android SoC Power & Performance Overview. Available: <https://www.anandtech.com/show/12195/hisilicon-kirin-970-power-performance-overview>

[14] Andrei Frumusanu. (2018). The Samsung Galaxy S9 and S9+ Review. Available: <https://www.anandtech.com/show/12520/the-galaxy-s9-review>

[15] Kang, Shin-haeng, Duseok Kang, and Hoeseok Yang, "Real-time co-

scheduling of multiple dataflow graphs on multi-processor systems,”
Proceedings of the 53rd Annual Design Automation Conference, pp.1-6,
2016.

Abstract

A Scheduling Technique for Multiple Deep Learning Applications on Heterogeneous Processors

Jinwoo Oh

School of Computer Science Engineering

The Graduate School

Seoul National University

The scheduling of multiple deep learning applications on heterogeneous processors is basically an NP-hard problem with a very large problem space. Meta-heuristics such as GAs (Genetic Algorithms) may be applied, but these have the disadvantage of having too long an execution time to be applied at run time. Therefore, this study proposes a new scheduling heuristic, which complements this shortcoming and does not significantly degrade scheduling performance. The proposed heuristic overcomes the limitations of traditional list scheduling techniques that fail to take into account the schedulability issue in the scheduling of multiple applications and introduces a new approach called ‘synthesis and iterative improvement’. It is confirmed through experiments with different deep learning networks on heterogeneous processors (including CPUs, GPUs, and NPU) that the proposed heuristic produces good scheduling results

that are sufficiently fast to apply at run time.

Key words: **embedded system, deep learning, heterogeneous processors, scheduling, heuristic**

Student Number: 2018-26822